Developing AI Apps and Agents on Azure

AI-103 - v1.1.0 - Practice with answers

10 questions · Thursday, October 8, 2026

Answer keys, explanations and source citations included.

DRAFT - independent content audit pending

Independent practice resource. Scores do not represent an official scaled score. Difficulty labels are provisional author estimates.

Exam: ai103-pilot-001-r1
Package revision: 1
Draft preview SHA-256: 122f5ce31e91e3055ef7d9e2ffe7ef841a9aafcba70b50ea6a3056da671782d6
Draft package SHA-256: 0ceafb2d5784ffa424193a00ff62ffb47731c6f902e9c766010db3fe582b358b
Objectives effective: 2026-04-16

Questions & Answers

Q1

Domain A: Plan and manage an Azure AI solution | case study child | medium

Case study - Case study: Cedar Atlas support agent

Cedar Atlas is a fictional equipment supplier building an internal support agent. Use this shared context for Q1 and Q2. Each question is independent; an answer to Q1 does not change the facts for Q2.

Inference and workload requirements

The agent answers interactive support requests. The approved processing boundary is the EU data zone; processing must not move outside it. Traffic is irregular, and the team wants pay-per-token billing without reserving throughput units. Asynchronous batch processing is unsuitable for the user-facing responses. Assume the chosen model supports all deployment types listed in Q1 and sufficient quota is available.

Retrieval and network requirements

Company manuals are indexed in Azure AI Search. Foundry Agent Service uses standard setup with private networking. Public access to Search must stay disabled. Private endpoint routing and DNS resolution have been verified from the relevant network. An authorized developer can query the index from that network.

Identity findings

The agent uses the documented Azure AI Search tool and a project connection to the correct index. The connection currently uses a Search API key. For this setup, the keyless connection principal is the system-assigned managed identity of the Foundry account containing the project. Search RBAC is enabled. That identity already has Search Service Contributor on Search, but lacks Search Index Data Contributor. The developer and deployment pipeline have provisioning permissions; neither is intended as the runtime connection principal. The agent's model calls already succeed.

Which model deployment type meets the case study's processing-boundary, billing and interactive-response requirements?

A Global Standard

B Data Zone Standard ✓

C Data Zone Provisioned

D Data Zone Batch

Why the correct answer is right

Data Zone Standard processes requests within the selected data zone and uses pay-per-token billing. It matches the approved EU boundary and an interactive workload without a reserved-throughput commitment.

Answer and option notes

- **A:** Global Standard uses global processing. Selecting an EU resource location does not convert the global processing boundary into the required EU data-zone boundary.
- **C:** Data Zone Provisioned respects the data-zone boundary but uses reserved provisioned throughput units, contrary to the stated billing requirement.
- **D:** Data Zone Batch respects the data-zone boundary but is for asynchronous batch workloads, which the case explicitly excludes for interactive responses.

AI103-A2.02

Sources

S005: Deployment types for Microsoft Foundry Models
https://learn.microsoft.com/azure/foundry/foundry-models/concepts/deployment-types
Snapshot SHA-256: 6c6d5d7b11684c1cbba8d35fd278a313e1420e3853b0ba9f104898c4a8c24c58

Q2

Domain A: Plan and manage an Azure AI solution | case study child | very_hard

Case study - Case study: Cedar Atlas support agent

Cedar Atlas is a fictional equipment supplier building an internal support agent. Use this shared context for Q1 and Q2. Each question is independent; an answer to Q1 does not change the facts for Q2.

Inference and workload requirements

The agent answers interactive support requests. The approved processing boundary is the EU data zone; processing must not move outside it. Traffic is irregular, and the team wants pay-per-token billing without reserving throughput units. Asynchronous batch processing is unsuitable for the user-facing responses. Assume the chosen model supports all deployment types listed in Q1 and sufficient quota is available.

Retrieval and network requirements

Company manuals are indexed in Azure AI Search. Foundry Agent Service uses standard setup with private networking. Public access to Search must stay disabled. Private endpoint routing and DNS resolution have been verified from the relevant network. An authorized developer can query the index from that network.

Identity findings

The agent uses the documented Azure AI Search tool and a project connection to the correct index. The connection currently uses a Search API key. For this setup, the keyless connection principal is the system-assigned managed identity of the Foundry account containing the project. Search RBAC is enabled. That identity already has Search Service Contributor on Search, but lacks Search Index Data Contributor. The developer and deployment pipeline have provisioning permissions; neither is intended as the runtime connection principal. The agent's model calls already succeed.

Which connection and authorization change addresses the documented retrieval problem while preserving the case study's network and runtime-identity requirements?

A Keep the key-based Search connection; grant Search Index Data Contributor to the developer.

B Authenticate the Search connection as the deployment pipeline; grant Search Index Data Contributor to that identity.

C Authenticate the Search connection with the Foundry account's system-assigned managed identity; grant it Foundry User only.

D Authenticate the Search connection with the Foundry account's system-assigned managed identity; grant it Search Index Data Contributor. ✓

Why the correct answer is right

The captured Search tool guide requires managed-identity authentication for private networking. Its keyless setup identifies the system-assigned identity of the Foundry account containing the project and requires both Search Service Contributor and Search Index Data Contributor on Search. The case already grants the first role and enables RBAC; D supplies the missing role and changes the connection to the documented identity. Provisioning access for a developer or pipeline does not substitute for the runtime connection's access.

Answer and option notes

- **A:** The captured tool guide does not support key-based authentication with private virtual networking. Granting a role to the developer also does not authorize the managed identity used by the connection.
- **B:** The documented keyless setup uses the containing Foundry account's system-assigned identity. The pipeline is a provisioning identity in this case, not the configured runtime principal.
- **C:** Foundry User is a project role; it does not supply the missing Search Index Data Contributor role on the Search service.

AI103-A3.04

Sources

S011: Set up private networking for Foundry Agent Service
https://learn.microsoft.com/azure/foundry/agents/how-to/virtual-networks
Snapshot SHA-256: 02eb40fb9bc5fcbd6b27249423b2d0ba26f5df0dd6b6885c881270a7a6fac8b9

S040: Connect an Azure AI Search index to Foundry agents
https://learn.microsoft.com/azure/foundry/agents/how-to/tools/ai-search
Snapshot SHA-256: 67830020b25de8c993ae294ff523035ad2c04c16865c1fe7222f053769316545

Q3

Domain B: Implement generative AI and agentic solutions | dropdown completion | very_hard

Complete the three settings for a supported model using strict structured outputs through the Responses API. Every successful, non-refusal structured output must contain order_id and note; when no note exists, note must be null. Extra properties must be disallowed. The property definitions below are already correct.

Existing property definitions (illustrative JSON Schema fragment)

```
{
  "type": "object",
  "properties": {
    "order_id": {
      "type": "string"
    },
    "note": {
      "type": [
        "string",
        "null"
      ]
    }
  }
}
```

Completed statement:

required: **["order_id", "note"]**

additionalProperties: **false**

Responses request format location: **text.format**

Why the correct answer is right

The strict schema includes both properties in required. The existing string-or-null type represents an absent note without omitting its key. additionalProperties must be false. For the Responses API, the strict JSON-schema format is configured under text.format; response_format is the Chat Completions location.

Answer and option notes

- **required:** Choose ["order_id", "note"]. Choosing only ["order_id"] omits note from the required list, and [] makes neither field required. The captured strict-schema subset requires all defined fields; nullable values provide optional semantics.
- **additional:** Choose false. true permits fields outside the declared contract and violates the documented strict-schema requirement. Nullability of note does not permit extra object properties.
- **format:** Choose text.format for Responses. response_format belongs to Chat Completions. JSON mode alone guarantees valid JSON without enforcing this schema.

AI103-B3.01

Sources

S037: Structured outputs
https://learn.microsoft.com/azure/foundry/openai/how-to/structured-outputs
Snapshot SHA-256: 0337c89ce3e5a9ba28d1c2fc18f1644c50b13568eb19edc43420d16b401cacfc

Q4

Domain B: Implement generative AI and agentic solutions | **log or artifact interpretation** | **hard**

Two function requests in a Responses flow call the same local function for different products. Both local executions succeed. Which change is required when returning the outputs shown in this illustrative trace?

Illustrative application trace, not a captured Azure error

```
Output strings are decoded below for readability; serialization is already correct.

Requested function calls:
  call_id=call_red  name=get_stock  arguments={"sku":"R-14"}
  call_id=call_blue name=get_stock  arguments={"sku":"B-27"}

Local execution results:
  call_red  -> {"sku":"R-14","available":0}
  call_blue -> {"sku":"B-27","available":8}

Current outgoing records:
  type=function_call_output call_id=get_stock output={"sku":"R-14","available":0}
  type=function_call_output call_id=get_stock output={"sku":"B-27","available":8}
```

A Keep get_stock as call_id and send the two outputs in the same order as the requests.

B Combine both results into one output and use get_stock as the identifier for the combined call.

C Return each result with its originating call_id: call_red for R-14 and call_blue for B-27. ✓

D Replace call_id with the function name in a tool_call_id field while retaining function_call_output.

Why the correct answer is right

A function_call_output must carry the originating function call's call_id. The name identifies the function, while call_red and call_blue distinguish these two invocations. Matching each returned result to its actual call preserves the association before the model continues.

Answer and option notes

- **A:** Output order does not replace the required call_id association. Both invocations share the same function name, so get_stock is not either originating call identifier.
- **B:** The model requested two distinct calls. Merging their results under the shared function name fails to return outputs associated with the two originating call IDs.
- **D:** The captured Responses example uses function_call_output with call_id. Substituting a differently named field from another API pattern does not satisfy that contract.

AI103-B1.03 AI103-B2.02

Sources

S007: Use function calling with Microsoft Foundry agents
https://learn.microsoft.com/azure/foundry/agents/how-to/tools/function-calling
Snapshot SHA-256: 0d4119f94b15c94d404729f53d2ed5f52a89dfaf35841d1290c0472be3e3e953

Q5

Domain B: Implement generative AI and agentic solutions | **single choice** | **medium**

A user asks an agent for quotes from two carriers and a comparison table with delivery dates. Traces show successful calls to both carriers with correct parameters. The agent follows its system policy against modifying records, but its final response includes only one quote and no comparison table. Which evaluator most directly measures this failure to deliver the requested outcome?

A Task Completion (preview) ✓

B Tool Call Success

C Task Adherence (preview)

D Groundedness

Why the correct answer is right

Task Completion evaluates whether the agent produced a usable deliverable meeting all user requirements. Successful carrier calls and compliance with the system policy do not establish that the requested two-carrier comparison was delivered.

Answer and option notes

- **B:** Tool Call Success measures technical success or errors in tool execution. Both calls succeeded here, so that measure does not directly capture the omitted final deliverable.
- **C:** Task Adherence measures compliance with assigned rules, procedures and policy constraints. The stated system policy was followed; the described failure is incomplete task delivery.
- **D:** Groundedness checks fidelity to supplied context. The single quote could be fully grounded while the response still omits the second quote and comparison table.

AI103-B2.06

Sources

S014: Agent evaluators
https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators
Snapshot SHA-256: 0ce17b86785e87efb44074de9d3fdee3ae8290dedcb04a18a68558081be7103f

S039: Retrieval-Augmented Generation (RAG) evaluators
https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/rag-evaluators
Snapshot SHA-256: 3b16c765be1f1479ac22e04cf9b293be767d473fc1cacb188b90751227074a4b

Q6

Domain C: Implement computer vision solutions | single choice | hard

An application uses the captured Sora 2 preview v1 interface to remix a previously completed video. The call returns the result below. Which next step lets the application display the requested edit?

Illustrative v1 remix request and result

```python
video = client.videos.remix(
    video_id="video_original_01",
    prompt="Change only the lighting to warm late-afternoon light."
)
```

```json
{
  "id": "video_remix_02",
  "status": "queued",
  "remixed_from_video_id": "video_original_01"
}
```

A Download the content for video_original_01, because a remix updates the original video in place.

B Track video_remix_02 until it completes successfully, then retrieve that new video's content. ✓

C Display the returned JSON as the completed media, because the remix call succeeded.

D Submit another videos.create request with the same prompt, because a queued remix cannot complete.

Why the correct answer is right

The remix returns a new video operation with its own ID and queued status. The application must track that operation through completion and retrieve its generated content. The original completed video is the remix input, not the newly edited output.

Answer and option notes

- **A:** The result identifies a new video and separately records remixed_from_video_id. Downloading the original ID retrieves the source video, not the requested edit.
- **C:** A successful creation response acknowledges the operation. queued is not a completed state, and the JSON record is status metadata rather than finished video content.
- **D:** Queued work is part of the documented asynchronous lifecycle. Submitting a new generation request does not complete or retrieve this remix and can create unnecessary work.

AI103-C1.04

Sources

S018: Video generation with Sora 2 (preview)
https://learn.microsoft.com/azure/foundry/openai/concepts/video-generation
Snapshot SHA-256: 32257da44487ca802a87e1ebad7ef31dc91f45f034d3334fb60b1c7d53ffb808

Q7

Domain A: Plan and manage an Azure AI solution | multi select | hard

Two groups of hosted-agent requests fail as described below. Required persisted session state must be retained, and policy forbids moving either workload to another region. Model token usage is below its quota. Which TWO actions address the reported failure layers without violating those requirements?

Illustrative error summary

```
Group P: HTTP 429, code=session_quota_exceeded
  Many sessions no longer need active compute; their required state must be retained.

Group R: HTTP 429, code=regional_session_quota_exceeded
  Retrying session creation in the approved region reports insufficient regional capacity.
```

A For group P, stop sessions that no longer need active compute without deleting required retained state. ✓

B For group P, delete all stored sessions so that neither active compute nor persisted state consumes capacity.

C For both groups, raise the model deployment's tokens-per-minute quota as the sole corrective action.

D For group R, route the workload immediately to a different region with spare session capacity.

E For group R, use bounded retries with exponential backoff and jitter within the approved region. ✓

Why the correct answer is right

Group P reports the concurrent session quota layer; the guide recommends stopping sessions that do not need active compute, deleting state only when no longer needed, or requesting a limit increase. Group R reports regional session capacity; bounded backoff with jitter is compatible with the prohibition on moving regions. The returned codes identify limits distinct from model tokens-per-minute.

Answer and option notes

- **B:** Deleting all stored sessions violates the explicit requirement to retain required persisted state. The documented choices distinguish stopping unneeded active compute from deleting sessions whose state is no longer needed.
- **C:** The errors identify session and regional capacity limits, not the model token quota. Increasing tokens-per-minute alone does not address those reported layers.

- **D:** Another region is a documented option in some circumstances, but the scenario explicitly forbids moving either workload outside the approved region.

AI103-A3.01

Sources

S009: Foundry Agent Service limits, quotas, and regional support
https://learn.microsoft.com/azure/foundry/agents/concepts/limits-quotas-regions
Snapshot SHA-256: d83fd890729d322d4ffa580ca5a8db2bed5bcd783dae47f5da2b05985f4b3041

Q8

Domain C: Implement computer vision solutions | multi select | medium

An app uses the captured Azure Content Safety image:analyze API contract (2024-09-01). Its policy checks all four harm categories, rejects an image when any category is Medium or High, and otherwise allows it. Which TWO implementation choices match that policy and the documented image API?

A Allow any image when the API returns HTTP success, because successful analysis means the content is approved.

B Reject when any returned category has severity 4 or 6; otherwise allow the image under the stated policy. ✓

C Request EightSeverityLevels for images and reject only severity values 5, 6 or 7.

D Request only the Violence category, because it includes the other three harm categories.

E Analyze all four categories using FourSeverityLevels, whose documented values are 0, 2, 4 and 6. ✓

Why the correct answer is right

The documented image moderation contract supports FourSeverityLevels: 0 Safe, 2 Low, 4 Medium and 6 High. Analyzing all four categories and rejecting any result at 4 or 6 implements the stated rule. HTTP success indicates successful analysis; the app still applies its acceptance policy.

Answer and option notes

- **A:** Successful analysis is not itself a content-approval decision. The result supplies category severities, and acceptance depends on the configured policy.
- **C:** The captured image API supports FourSeverityLevels, not the proposed eight-level output. The proposed threshold also fails to follow the stated Medium-or-High rule for the documented values.
- **D:** Hate, SelfHarm, Sexual and Violence are separate categories. Requesting only Violence omits signals that the policy explicitly requires.

AI103-C3.01

Sources

S042: QuickStart: Analyze image content
https://learn.microsoft.com/azure/ai-services/content-safety/quickstart-image
Snapshot SHA-256: 6a40b6fdabe9e2a91838a3026f8530d0229c70acd401c4fab094a6c0c63b8670

Q9

Domain D: Implement text analysis solutions | sequence order | medium

Arrange these steps for a Python Speech SDK workflow that translates one Italian utterance into German text and then synthesizes that translated text with a separately configured German voice. Credentials, a supported input device and the German synthesis voice are already available. Assume the recognition/translation succeeds.

Correct order:

1. Create SpeechTranslationConfig, set the recognition locale to it-IT, add de as a target language, and create the audio input configuration.
2. Construct TranslationRecognizer with the completed translation configuration and the audio input configuration.
3. Invoke recognize_once() on the recognizer and confirm that the returned reason is TranslatedSpeech.
4. Read the German text from result.translations["de"] and pass that text to the separately configured German SpeechSynthesizer.

Why the correct answer is right

The recognizer depends on the configuration, recognition/translation depends on the recognizer, and text synthesis depends on the successful translation result. The source recognition locale it-IT and translation target de have distinct roles. A German voice alone does not translate Italian input.

Answer and option notes

- **A:** This follows C because TranslationRecognizer consumes the configured translation and audio objects. It must precede the recognition call in D.
- **B:** This comes last because the German text is obtained from the successful translation result. Synthesizing before that result is available would not use the requested translated text.
- **C:** This comes first because the source locale, target language and input must be defined before the recognizer is constructed.
- **D:** This follows recognizer construction and precedes extraction/synthesis. The result reason confirms successful speech translation before its translations dictionary is used.

AI103-D2.04

Sources

S046: How to recognize and translate speech
https://learn.microsoft.com/azure/ai-services/speech-service/how-to-translate-speech
Snapshot SHA-256: 8a73fb1e63f49dde972815b860758a57ff3d620842220f9e29200bccf573c39e

Q10

Domain E: Implement information extraction solutions | matching magnet | hard

Match each Azure AI Search ingestion requirement to the component that directly performs it. Choose one component per row; each component may be used at most once, and one choice is unused. All target index fields and required service resources already exist.

Correct matches:

Copy an unchanged metadata_storage_name field from the data source into the index field file_name.	**→ fieldMappings**
Copy OCR text already produced at an enrichment-tree path into the target searchable index field.	**→ outputFieldMappings**
Analyze document structure and emit a Markdown or text representation that preserves layout information.	**→ Document Layout skill**
Read characters from the indexer's normalized image inputs and emit extracted text.	**→ OCR skill**

Choices: outputFieldMappings, OCR skill, Query-time vectorizer, fieldMappings, Document Layout skill

Why the correct answer is right

fieldMappings maps source fields; outputFieldMappings maps enriched document nodes into index fields. The Document Layout skill uses Document Intelligence layout analysis to produce structured Markdown or text. The OCR skill reads normalized image inputs and emits extracted text. A query-time vectorizer serves query embedding, not these ingestion or field-routing tasks.

Answer and option notes

- **A:** Use fieldMappings because the value is an unchanged source field. outputFieldMappings is for a node produced in the enrichment tree, which this row does not require.
- **B:** Use outputFieldMappings because OCR has already produced an enriched node. Re-running OCR or changing a source-field mapping does not perform the required enriched-node-to-index routing.
- **C:** Use Document Layout skill because the requirement is structural analysis and Markdown/text output. OCR focuses on character extraction from images and does not replace the stated layout-analysis contract.
- **D:** Use OCR skill because normalized images must be converted into text. The source and output mappings route data but do not themselves recognize characters.

AI103-E1.03

Sources

S028: Skillset concepts in Azure AI Search
https://learn.microsoft.com/azure/search/cognitive-search-working-with-skillsets
Snapshot SHA-256: 3d24961260a3a7ca041656383016222704c25dec2dff53743bbaef628236312b

S029: Document Layout skill
https://learn.microsoft.com/azure/search/cognitive-search-skill-document-intelligence-layout
Snapshot SHA-256: eb899b4d8c7fdd95e158072fa95891c8c96388f09c48c00a67683fa10ec4591a

S043: OCR cognitive skill
https://learn.microsoft.com/azure/search/cognitive-search-skill-ocr
Snapshot SHA-256: 0a5cdd696dcbae744d32db7fd89a69926f0203462f86d6dc25793ab07d001b3d

S026: Integrated vector embedding in Azure AI Search
https://learn.microsoft.com/azure/search/vector-search-integrated-vectorization
Snapshot SHA-256: 45a6b445fbe4f7f65334693eb96b2de7b021e0b50ce46b363ab47c5de0a11f0d

Quick Answer Key

#	Type	Correct Answer
Q1	case study child	**B**
Q2	case study child	**D**
Q3	dropdown completion	**required → ["order_id", "note"] | additional → false | format → text.format**
Q4	log or artifact interpretation	**C**
Q5	single choice	**A**
Q6	single choice	**B**
Q7	multi select	**A, E**
Q8	multi select	**B, E**
Q9	sequence order	**C → A → D → B**
Q10	matching magnet	**A → fieldMappings | B → outputFieldMappings | C → Document Layout skill | D → OCR skill**